

Arka Warehouse Management System – Complete Documentation

This document outlines the architecture, features, and technical specifications of the Arka Warehouse Management System (WMS), a full-scale platform for multi-warehouse operations, inventory tracking, and custom package management.

Table of Contents

- Overview
- System Architecture
- Core Features
- User Roles and Permissions
- Feature Details
- API Endpoints
- Data Models
- Technical Stack
- Getting Started
- Common Use Cases
- Support and Troubleshooting
- Version History
- Arka Architecture Documentation

Overview

This is a full-scale **warehouse management platform** that supports multi-warehouse operations, **inventory tracking**, custom package management, and a **visual notification workflow builder**. It includes user management, admin tools, product catalog management, and a modern interface with light and dark themes.

Key capabilities include:

- **Multi-warehouse support**
- **Product and inventory tracking**
- **Visual notification workflows**
- User and admin management
- Real-time dashboard metrics
- Full authentication system
- Dark and light themes
- Notification workflows with triggers and channels

System Architecture

Frontend

Component	Technology	Purpose
Framework	React with TypeScript	Core UI development
Routing	Wouter	Client-side navigation
Data Fetching	TanStack React Query	State management and caching
Validation	React Hook Form + Zod	Form handling and schema validation
Styling	Tailwind CSS + shadcn UI	Component styling and library
Theming	Theme provider using localStorage	Dark/light mode persistence

Backend

Component	Technology	Purpose
Framework	Express.js with TypeScript	Server foundation and API endpoints
Authentication	Passport.js	User session management and security

Database	PostgreSQL (Neon)	Primary data storage
ORM	Drizzle ORM	Database query abstraction
Sessions	Express sessions with PostgreSQL store	Session data persistence

Database

Relational model with **13 primary tables**, covering users, organizations, warehouses, products, inventory, packages, notifications, and more.

Core Features

Authentication

- Email-based registration
- Username and email uniqueness checks
- Password hashing (**bcrypt**, 12 rounds)
- **Passport session authentication**
- Enable and disable user accounts

Stored user data includes ID, username, name, email, admin flag, enabled flag, timestamps, and organization ID.

Dashboard

Displays key statistics:

- Total customers
- Total products
- Total warehouses
- Total orders

Includes a recent activity feed, user menu, notification bell, and personalized greeting.

Warehouse Management

User Type	Capabilities
All Users	Create, edit, delete personal warehouses , manage warehouse inventory, view warehouse details
Admins	Full view of all warehouses, bulk operations, advanced search and filtering

Product Catalog

- Search and filter products
- Paginated browsing
- Admins can create, update, delete products
- Products include dimensions, category, and model number

Inventory Management

- Track product quantities **per warehouse**
- Set **low stock thresholds**
- Real-time updates
- Per warehouse isolation

Custom Packaging

Users can define custom packages with name, dimensions, weight, cost, and associated customer.

Notification Workflow System

A **visual workflow builder** with drag-and-drop events and channels.

- **Events:** Low Stock, New Product, Order Fulfilled, Restock
- **Channels:** Email, SMS, Slack

Workflows are saved to the database and tied to users and warehouses.

User Management (Admin)

- View all users
- Enable or disable accounts
- Manage admin privileges
- View login timestamps

Theme Support

- **Dark and light mode**
- Automatic persistence
- Smooth UI transitions

Unit Conversion

- Metric and imperial conversions for dimensions and weight
- Default system is **imperial**

User Roles and Permissions

Role	Can Access	Cannot Access
Regular User	Dashboard, Their warehouses, Product catalog, Warehouse inventory, Notification customizer	Admin warehouses, Admin users
Admin User	Full access , including All warehouses, All users, Product catalog management, Global organizational control	N/A

Feature Details

Flow	Steps
Registration Flow	User fills registration form → Zod validates input → Check for duplicate username and email → Password hashed → User created and logged in
Login Flow	User enters email and password → Password verified via bcrypt → Session created → User redirected to dashboard

Warehouse Creation	User selects Create Warehouse → Inputs name and address → Warehouse is stored and linked to organization and user
Inventory Addition	User opens warehouse inventory → Selects product or custom package → Inputs quantity and threshold → Inventory entry created
Notification Workflow Setup	User drags event onto canvas → Drags channel onto canvas → Connects nodes → Saves workflow

API Endpoints

Below is the cleaned list of all endpoints.

Resource	HTTP Method	Endpoint	Permissions
Authentication	POST	/api/register	Public
	POST	/api/login	Public
	POST	/api/logout	Authenticated
	GET	/api/user	Authenticated
Warehouses	GET	/api/warehouses	Authenticated
	GET	/api/warehouses/:warehouseId	Authenticated

	POST	/api/warehouses	Authenticated
	PATCH	/api/warehouses/:warehouseId	Authenticated
	DELETE	/api/warehouses/:warehouseId	Authenticated
	GET	/api/admin/warehouses	Admin Only
Product Catalog	GET	/api/admin/catalogue	Admin Only
	GET	/api/admin/catalogue/:id	Admin Only
	POST	/api/admin/catalogue	Admin Only
	PATCH	/api/admin/catalogue/:id	Admin Only
	DELETE	/api/admin/catalogue/:id	Admin Only
Inventory	GET	/api/warehouse/:id/inventory	Authenticated
	POST	/api/warehouse/:id/inventory	Authenticated
	PATCH	/api/warehouse/:id/inventory/:invId	Authenticated

	DELETE	/api/warehouse/:id/inventory/:invId	Authenticated
Notifications	GET	/api/notifications	Authenticated
	GET	/api/notifications/unread	Authenticated
	PATCH	/api/notifications/:id	Authenticated
	PATCH	/api/notifications/read-all	Authenticated
	POST	/api/notifications	Authenticated
Custom Packages	GET	/api/packages	Authenticated
	POST	/api/packages	Authenticated
User Management	GET	/api/admin/users	Admin Only
	PATCH	/api/admin/users/:userId	Admin Only

Data Models

Model	Key Fields
Users	id, username, firstName, lastName, email, password , enabled , arkaAdmin , createdAt, lastLogin, organizationId
Warehouses	id, name, address fields, active status, organizationId , createdBy, createdAt
Product Catalogue	id, modelNo, dimensions, category, lastUpdated
Warehouse Inventory	id, warehouseId , productCatalogueId , quantity , lowStockThreshold , createdAt, updatedAt
Warehouse Products	id, warehouseId, name, dimensions, weight, imageUrl, createdBy, timestamps
Notifications	id, userId, warehouseId, productCatalogueId, type, message, isRead , createdAt
Packages	id, name, dimensions, weight, cost, customer, createdAt, organizationId , createdBy
Organizations	id, name, address fields, logo

Technical Stack

Category	Technologies Used
Frontend	React, TypeScript , Vite, Tailwind , shadcn UI, Zod, React Query , Wouter, React Flow
Backend	Express, TypeScript, Passport, bcrypt, Drizzle ORM, PostgreSQL , Express Sessions
Tools	tsx, PostCSS, Tailwind utilities

Getting Started

Prerequisites

- Node 16+
- PostgreSQL database

Installation

1. Clone repository
2. npm install
3. Configure environment variables
4. Run migrations
5. npm run dev

Production

1. npm run build
2. npm run start

Common Use Cases

- Onboard new organization
- Monitor low stock levels
- Manage multiple warehouses
- Create custom packaging for shipping

Support and Troubleshooting

Issue	Resolution
Login issues	Verify email and enabled status
Warehouse missing	Refresh or check permissions
Notifications not triggering	Verify workflow connections

Version History

Version	Description
1.0.0	Initial release with all core features

Architecture Documentation

High Level Architecture

Client (React) → Express Server → Storage Layer → PostgreSQL Database

Frontend Architecture

React components, hooks, pages, and state via **React Query**.

Backend Architecture

Express handlers, **authenticated routes**, and storage abstraction.

Database Architecture

Relational schema with **foreign keys** and timestamp tracking.

Authentication Flow

Registration and login handled by **Passport**, sessions stored in **PostgreSQL**.

Data Flow

Frontend triggers **React Query**, backend validates, **Drizzle** executes queries, responses cached.

Directory Structure

client/src for frontend, server folder for backend, shared folder for schemas and utilities.